DOCUMENTAÇÃO TÉCNICA DO PROJETO ECO MOBILITY - SOULUP

Disciplina: Domain Driven Design Using Java
Projeto: Eco Mobility
Instituição: Faculdade de Informática e Administração Paulista
Equipe: SoulDevs

Integrantes:
Arthur Arruda da Silva - RM 569593
Enzo Torres Estanguera - RM 568937
Giovane Dias Chicoli - RM 569750
Álvaro Morais Holanda - RM 568607
Dhiego Gozzo Belmonte - RM 571505

SUMÁRIO

1. OBJETIVO E ESCOPO DO PROJETO
2. DESCRIÇÃO DAS FUNCIONALIDADES
3. ENDPOINTS DA API REST
4. PROTÓTIPO DO SISTEMA
5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)
6. DIAGRAMA DE CLASSES E ARQUITETURA DO SISTEMA
7. PROCEDIMENTOS PARA EXECUÇÃO DA APLICAÇÃO

1. OBJETIVO E ESCOPO DO PROJETO

1.1 Propósito Principal

O Eco Mobility SoulUp é uma solução desenvolvida em Java com o objetivo de incentivar hábitos sustentáveis relacionados à mobilidade urbana. A aplicação utiliza gamificação para estimular os usuários a realizar missões ecológicas, acumular pontos e trocá-los por vouchers.

O sistema disponibiliza uma API RESTful desenvolvida com Spring Boot, responsável por expor os recursos da aplicação e permitir a comunicação com clientes externos, como uma interface web. A persistência dos dados é realizada por meio de conexão JDBC com o banco de dados Oracle.

A solução também contempla informações relacionadas às missões sustentáveis, incluindo estimativas de redução de CO_2, contribuindo para que o usuário acompanhe o impacto ambiental de suas ações.

1.2 Escopo do Sistema

As principais atividades contempladas pelo sistema são:

- Cadastro, consulta, atualização e exclusão de usuários, conforme os endpoints implementados.
- Autenticação de usuários por e-mail e senha.
- Consulta de missões sustentáveis disponíveis.
- Registro da conclusão de missões, conforme as regras de negócio implementadas.
- Atribuição de pontos pelas missões concluídas.
- Consulta de vouchers disponíveis para resgate.
- Resgate de vouchers utilizando os pontos acumulados.
- Registro e consulta do histórico de resgates.
- Consulta de informações relacionadas à pontuação e à sustentabilidade.
- Disponibilização dos recursos por meio de endpoints REST.
- Acesso à API publicada na nuvem.

Fora do escopo

Não fazem parte do escopo atual, salvo se forem implementados posteriormente:

- Rastreamento de deslocamentos por GPS em tempo real.
- Monitoramento automático dos meios de transporte utilizados.
- Processamento de pagamentos em dinheiro.
- Integração com gateways de pagamento.
- Integração direta com aplicativos de transporte público.

A API pode ser consumida por uma aplicação web, mas a existência de uma interface web

integrada deve ser comprovada separadamente.

2. DESCRIÇÃO DAS FUNCIONALIDADES

2.1 Gestão de Usuários

A API disponibiliza recursos para operações relacionadas aos usuários, incluindo cadastro e autenticação. Os dados de usuário incluem nome, e-mail, senha e saldo de pontos. A senha não deve ser retornada nas respostas da API.

2.2 Catálogo de Missões Sustentáveis

O sistema disponibiliza missões relacionadas a práticas sustentáveis de mobilidade. Cada missão pode conter nome, descrição, pontuação, tipo de transporte, estimativa de CO_2, quantidade de conclusões e informações complementares.

A consulta das missões é realizada por meio dos endpoints disponibilizados pela API.

2.3 Conclusão de Missões

A aplicação disponibiliza uma operação para registrar a conclusão de uma missão associada a um usuário. Essa operação pode atualizar a pontuação e os dados relacionados à missão, conforme as regras implementadas no sistema.

2.4 Consulta de Pontos

Os pontos representam as recompensas acumuladas pelo usuário. Eles podem ser consultados por meio dos recursos de usuário e utilizados nas operações de resgate de vouchers.

2.5 Resgate de Vouchers

A API disponibiliza a consulta dos vouchers existentes e de seus respectivos custos em pontos. O resgate utiliza os pontos acumulados pelo usuário e deve respeitar as validações previstas nas regras de negócio.

2.6 Histórico de Resgates

O sistema permite consultar registros de resgates anteriores, incluindo o código do voucher, os pontos utilizados e a data da operação, conforme os dados persistidos no banco Oracle.

2.7 Trajetória Ecológica

A solução contempla informações associadas à pontuação, às missões concluídas e à estimativa de CO_2 economizado. A apresentação dessas informações em uma interface visual depende do cliente que

consome a API.

3. TABELA DE ENDPOINTS DA API RESTful

A API do Eco Mobility SoulUp foi desenvolvida com Spring Boot e disponibilizada na nuvem por meio da plataforma Render.

URL base: https://soulup-api-78mu.onrender.com

A tabela apresenta os principais recursos REST implementados na aplicação e seus respectivos métodos HTTP, URIs e códigos de resposta de sucesso.

Recurso	Método HTTP	URI	Resposta de sucesso
Verificar status da API	GET	`/api/status`	200 OK
Listar usuários	GET	`/api/usuarios`	200 OK
Consultar usuário por e-mail	GET	`/api/usuarios/{email}`	200 OK
Cadastrar usuário	POST	`/api/usuarios`	201 Created
Atualizar usuário	PUT	`/api/usuarios/{email}`	200 OK
Excluir usuário	DELETE	`/api/usuarios/{email}`	204 No Content
Realizar login	POST	`/api/usuarios/login`	200 OK

Listar missões	GET	`/api/missoes`	200 OK
Consultar missão por nome	GET	`/api/missoes/{nome}`	200 OK
Cadastrar missão	POST	`/api/missoes`	201 Created
Atualizar missão	PUT	`/api/missoes/{nome}`	200 OK
Excluir missão	DELETE	`/api/missoes/{nome}`	204 No Content
Registrar conclusão de missão	POST	`/api/missoes/{nome}/conclusoes?email={email}`	200 OK
Listar vouchers	GET	`/api/vouchers`	200 OK
Consultar voucher por código	GET	`/api/vouchers/{codigo}`	200 OK
Cadastrar voucher	POST	`/api/vouchers`	201 Created
Atualizar voucher	PUT	`/api/vouchers/{codigo}`	200 OK
Excluir voucher	DELETE	`/api/vouchers/{codigo}`	204 No Content

Resgatar voucher	POST	`/api/vouchers/{codigo}/resgates?email={email}`	200 OK
Consultar histórico por e-mail	GET	`/api/historicos?email={email}`	200 OK
Registrar histórico	POST	`/api/historicos?email={email}`	201 Created
Atualizar histórico por código	PUT	`/api/historicos/{codigo}?email={email}`	200 OK
Excluir histórico por código	DELETE	`/api/historicos/{codigo}?email={email}`	204 No Content

Os códigos de sucesso apresentados correspondem aos retornos definidos nas classes Resource. Dependendo da operação, também podem ocorrer respostas de erro, como `400 Bad Request`, `401 Unauthorized`, `404 Not Found`, `409 Conflict` e `500 Internal Server Error`.

3.1 Acesso à API publicada

A API está hospedada em:

https://soulup-api-78mu.onrender.com

Os seguintes endereços podem ser utilizados para consultar recursos disponíveis:

- **Status:** https://soulup-api-78mu.onrender.com/api/status
- **Missões:** https://soulup-api-78mu.onrender.com/api/missoes
- **Vouchers:** https://soulup-api-78mu.onrender.com/api/vouchers
- **Usuários:** https://soulup-api-78mu.onrender.com/api/usuarios

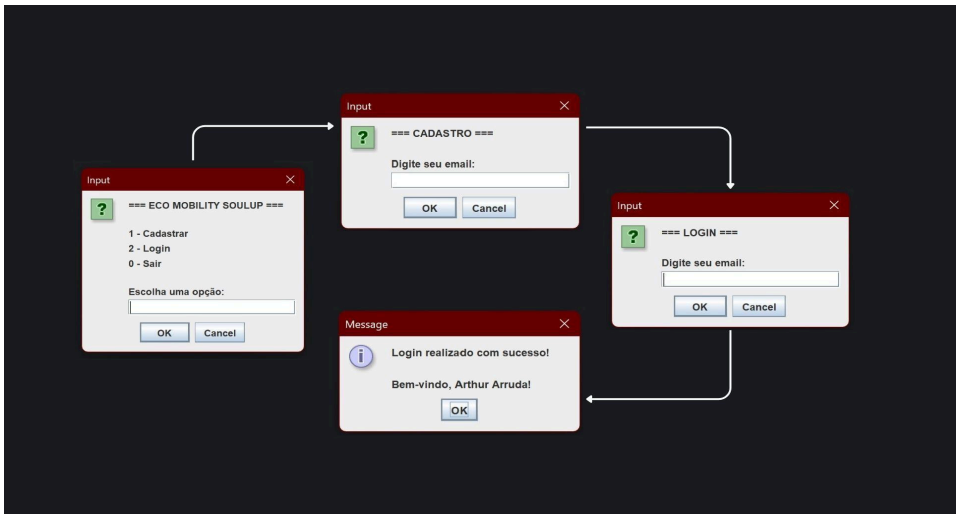
As requisições que modificam dados devem ser realizadas por um cliente HTTP, respeitando os métodos, parâmetros e corpos definidos pela API.

4. PROTÓTIPO DO SISTEMA

O projeto Eco Mobility SoulUp possui registros visuais de um protótipo desenvolvido anteriormente em Java, utilizando caixas de diálogo por meio da classe `JOptionPane`. O protótipo foi utilizado para representar o fluxo de navegação e as principais funcionalidades propostas para a solução.

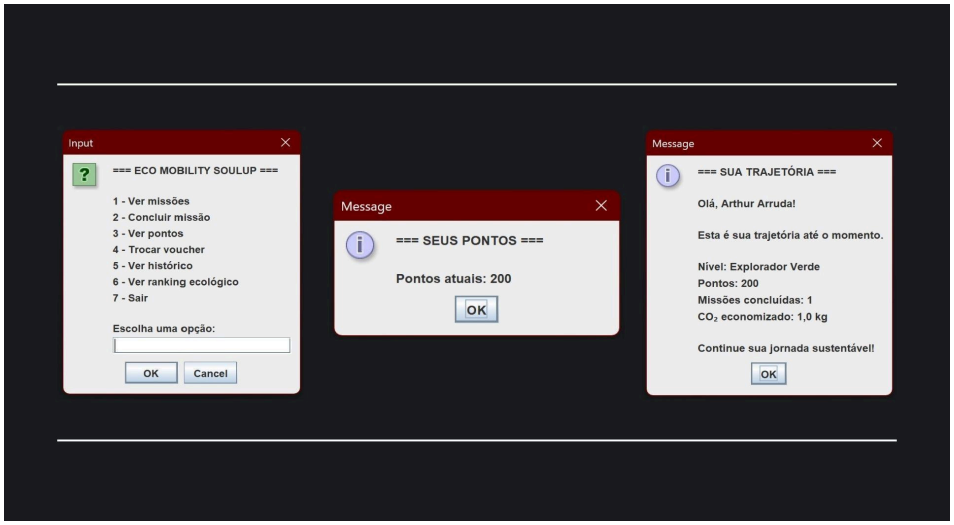
Na versão atual, a aplicação Java foi estruturada como uma API RESTful utilizando Spring Boot. Dessa forma, as imagens apresentadas nesta seção documentam o protótipo anterior e não representam, necessariamente, a interface da API atualmente publicada.

Figura 1 – Cadastro e Login



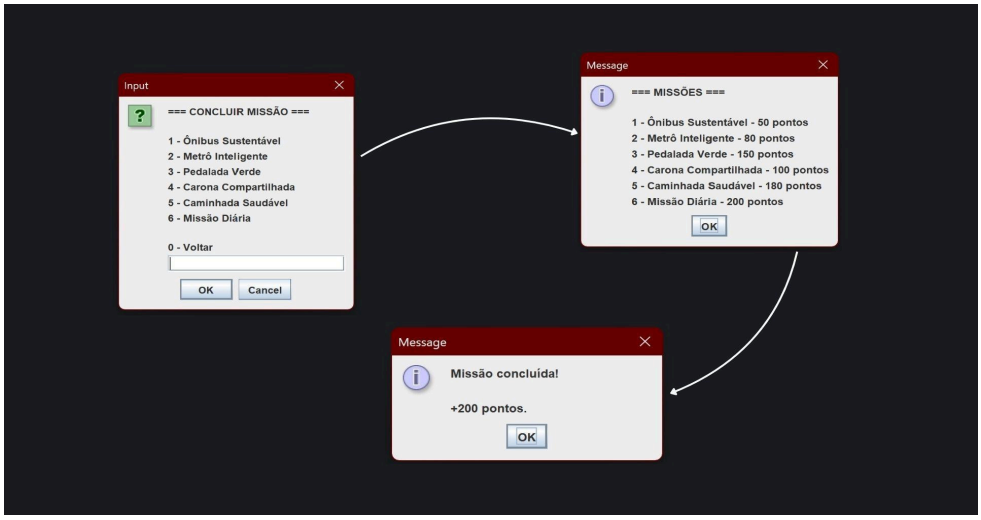
Descrição: o protótipo apresenta o menu inicial, com opções de cadastro e autenticação de usuários. Após a autenticação, o sistema informa o resultado da tentativa de acesso.

Figura 2 – Menu Principal, Pontuação e Trajetória Ecológica



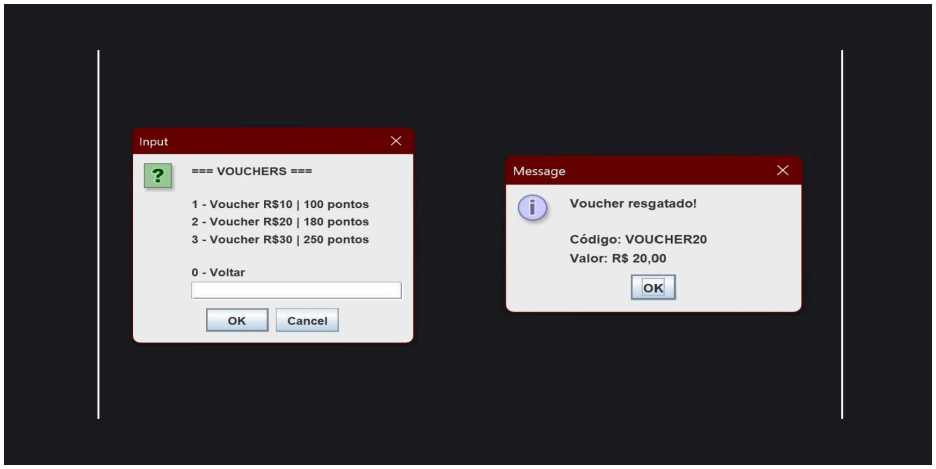
Descrição: o protótipo apresenta o menu principal e as opções para consultar a pontuação e a trajetória ecológica do usuário, incluindo seu nível de sustentabilidade, missões concluídas e estimativa de CO_2 economizado.

Figura 3 – Consulta e Conclusão de Missões



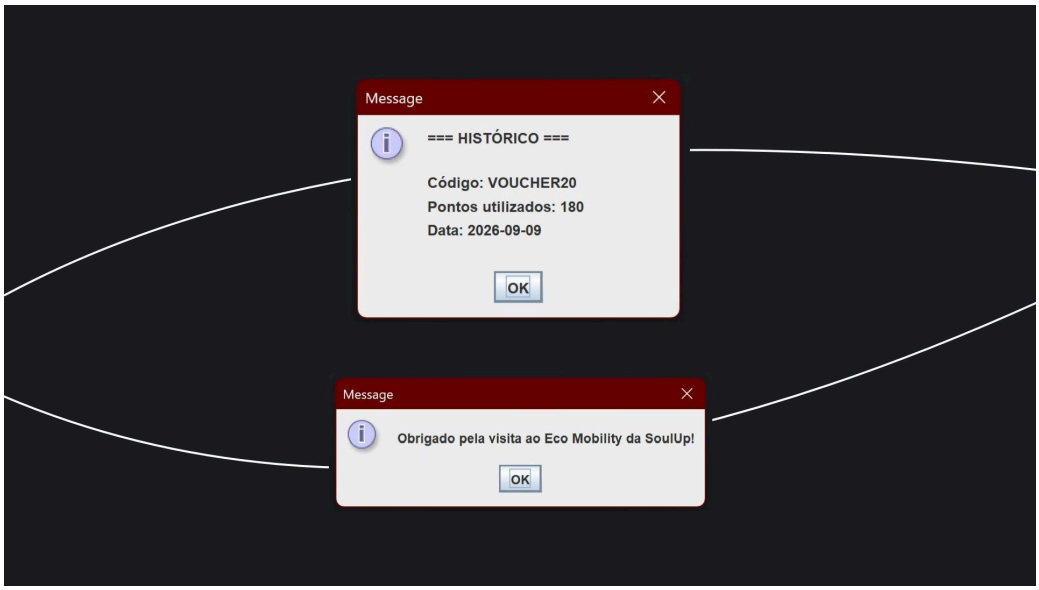
Descrição: o protótipo demonstra a consulta das missões sustentáveis e o fluxo de seleção e confirmação de uma missão, com a correspondente atribuição de pontos prevista pelas regras do sistema.

Figura 4 – Catálogo e Resgate de Vouchers



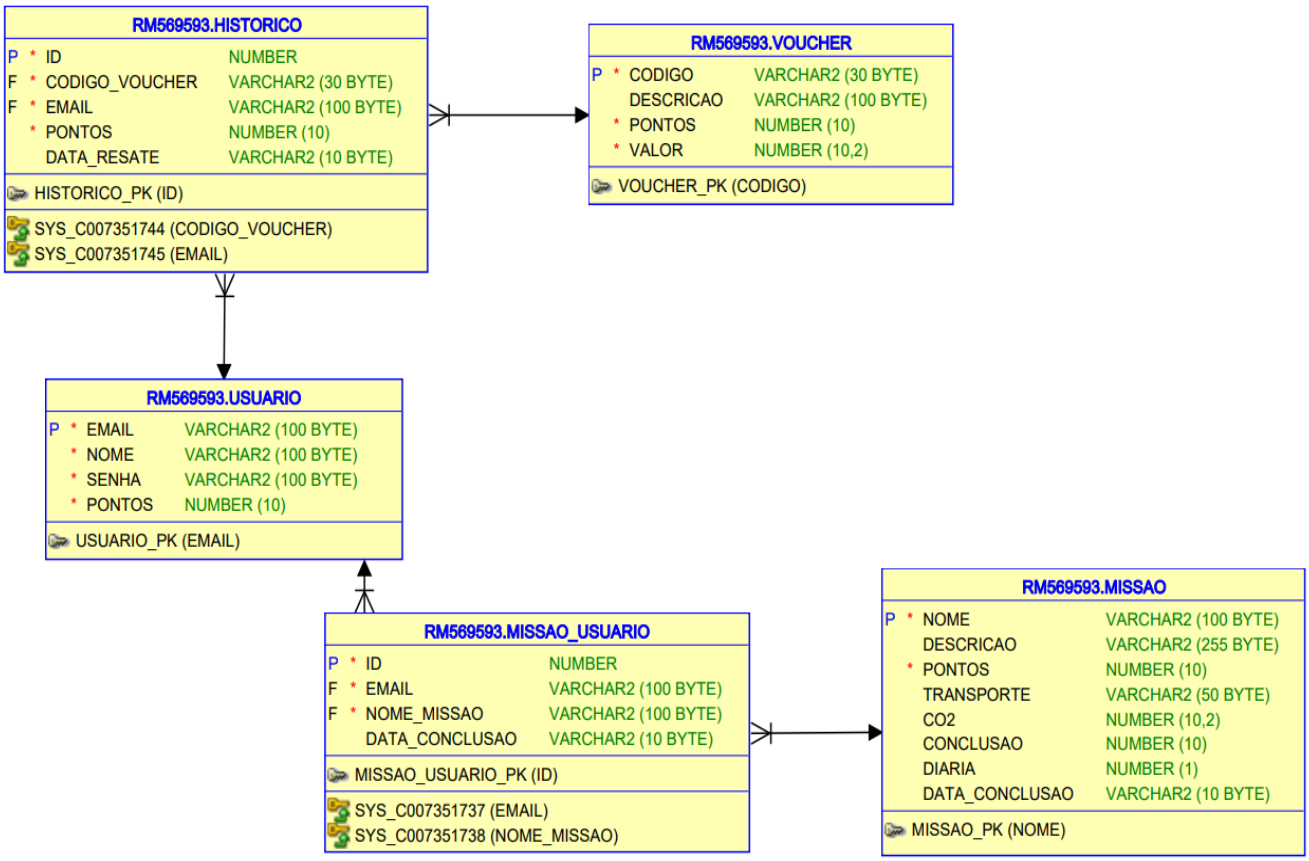
Descrição: o protótipo apresenta as opções de vouchers e seus custos em pontos, ilustrando o fluxo de seleção e confirmação de um resgate.

Figura 5 – Histórico de Resgates



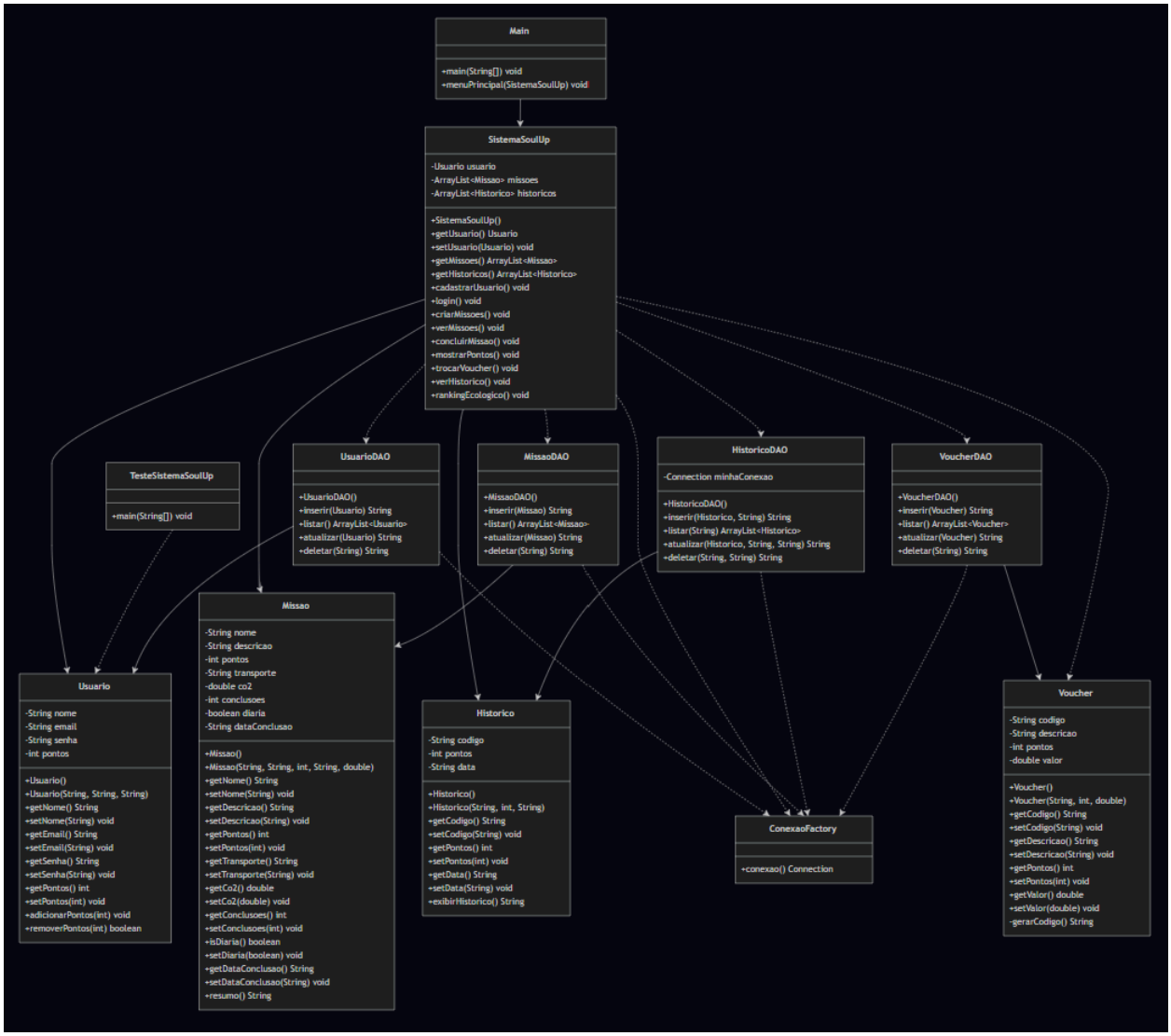
Descrição: o protótipo apresenta o histórico de vouchers resgatados, com informações como código, pontos utilizados e data da operação.

5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)



- USUARIO → MISSAO_USUARIO
- MISSAO → MISSAO_USUARIO
- USUARIO → HISTORICO
- VOUCHER → HISTORICO

6. DIAGRAMA DE CLASSES E ARQUITETURA DO SISTEMA



O projeto Eco Mobility SoulUp foi inicialmente estruturado em Java com uma arquitetura organizada em classes de entidade, classes DAO e controle da aplicação. Na versão atual, o sistema foi adaptado para uma API RESTful desenvolvida com Spring Boot, mantendo a integração com o banco de dados Oracle por meio de JDBC.

As principais entidades do domínio são Usuario, Missao, Voucher e Historico, responsáveis por representar os dados e conceitos centrais da aplicação. As classes Resource disponibilizam os endpoints REST e recebem as requisições HTTP, enquanto as classes DAO são responsáveis pelas operações de persistência e consulta no banco de dados.

A classe ConexaoFactory gerencia a configuração da conexão com o banco Oracle por meio das variáveis de ambiente definidas para a aplicação. A inicialização da versão atual é realizada pela classe principal SoulUpApplication, responsável por iniciar o contexto do Spring Boot e disponibilizar os recursos da API.

O diagrama de classes apresentado nesta seção pertence à estrutura anterior do projeto e é mantido como referência da modelagem inicial, incluindo atributos, métodos e relacionamentos entre as classes. Portanto, ele não representa integralmente a arquitetura da versão atual publicada na nuvem.

A classe TesteSistemaSoulUp também pertence à estrutura anterior e foi utilizada para testar métodos das entidades e simular operações do sistema. A implementação atual deve ser compreendida em conjunto com os recursos REST, as classes de persistência e a configuração do Spring Boot.

7. PROCEDIMENTOS PARA EXECUÇÃO DA APLICAÇÃO

7.1 Ferramentas e tecnologias utilizadas

- Java JDK 21;
- Spring Boot 3.4.5;
- IntelliJ IDEA;
- Apache Maven;
- Oracle Database;
- Oracle JDBC Driver;
- Oracle SQL Developer, para consulta e administração do banco de dados;
- Render, para hospedagem da API;
- Docker, para empacotamento e implantação da aplicação.

7.2 Configuração do ambiente

Para executar a aplicação localmente, é necessário ter o JDK 21 instalado e configurar o projeto no IntelliJ IDEA. O projeto utiliza Maven para gerenciar dependências e realizar a compilação.

As configurações de conexão com o banco Oracle são obtidas pelas variáveis de ambiente `SOULUP_DB_URL`, `SOULUP_DB_USER` e `SOULUP_DB_PASSWORD`. Antes de executar a aplicação, essas variáveis devem estar configuradas corretamente.

A URL de conexão deve apontar para o banco Oracle utilizado pelo projeto. O usuário do banco precisa ter permissão para acessar as tabelas necessárias à aplicação.

7.3 Execução da API

1. Abrir o projeto no IntelliJ IDEA.

2. Verificar se o projeto está configurado para utilizar o Java JDK 21.
3. Conferir as dependências declaradas no arquivo `pom.xml`.
4. Configurar as variáveis de ambiente necessárias para a conexão com o banco Oracle.
5. Localizar a classe principal `br.com.fiap.SoulUpApplication`.
6. Executar a aplicação Spring Boot.
7. Após a inicialização, acessar `http://localhost:8080/api/status` para verificar se a API está respondendo.

7.4 Acesso à aplicação publicada

A API também está hospedada na plataforma Render e pode ser acessada pelo endereço:

https://soulup-api-78mu.onrender.com

O endpoint de status pode ser utilizado para verificar a disponibilidade do serviço:

https://soulup-api-78mu.onrender.com/api/status

A API disponibiliza recursos REST para consulta de usuários, missões, vouchers e histórico, conforme os endpoints implementados no projeto.

As operações que alteram dados devem ser realizadas por meio de requisições HTTP apropriadas, respeitando os métodos, parâmetros e corpos definidos pela API.